SECTION 1

The blueprint

- **Destination diagram:** Figure 9.1 maps each box to a service already built; application code is the small workflow at the top.
- **Incremental delivery:** Each section introduces a limitation the current Claw cannot meet, then fixes it with one platform capability rather than a rewrite.
- **Channels, one route:** Slack and web clients share an `api_path` through the gateway so identity, trace, and session ids stay consistent.
- **Named assets up front:** Prompt registry name, indexes, tool namespace, guardrail policies, and sync versus stream should be explicit before coding.
- **Sarah's scope:** She composes calls and policies; she does not operate vector stores, credential vaults, or trace stores herself.

SECTION 2

The simplest model-call assistant

- **v1 shape:** Register `claw-assistant` prompt, decorate `handle_message` with `@workflow`, call `platform.models.chat`, return text.
- **Platform-owned leaf complexity:** Retries, routing, fallback, and usage accounting live in the Model Service, not in the handler.
- **Credibility gap:** Without stored turns, every message is isolated—"what you said last week" is impossible regardless of how articulate the model sounds.
- **Registry prompt:** Versioned system instructions can change without redeploying when prompts are targets, not string literals in code.

SECTION 3

Conversation memory and long-term memory

- **Session continuity:** `get_or_create`, `get_messages`, and `add_messages` give multi-turn dialogue that survives restarts and replica moves.
- **Window limits:** A capped history (sliding window) prevents context overflow but drops early turns unless something else remembers them.
- **Long-term facts:** Key-value memories persist across sessions; a memory tool lets the model decide what preferences and project details merit storage.
- **Two session bundles:** Recent transcript plus compact user memories both feed generation; memories are not corporate documents and should not live only in RAG indexes.

SECTION 4

Knowledge retrieval

- **Organizational grounding:** Data Service search supplies chunks so HR and engineering answers cite real policy text, not fluent pretraining guesses.
- **Pipeline RAG first:** Always-on search with score thresholds avoids injecting noise on casual chit-chat but fails on context-dependent queries like "what did engineering commit?"
- **Agentic RAG:** Register `claw.knowledge.search` so the model reformulates queries, picks indexes, and iterates when scores are weak.
- **Ingestion upfront:** Indexes, embeddings, chunking, and metadata must exist before the assistant can ground claims in live documents.
- **Source attribution:** Chunk labels and prompt instructions encourage citations users can verify.

SECTION 5

Tools and the agent loop

- **Managed tools:** Calendar, tickets, web search, and knowledge tools register under a `claw` namespace with credentials and rate limits outside application code.
- **ReAct loop:** The model may emit tool calls; the workflow executes via Tool Service, appends observations, and calls the model again until text or an iteration cap.
- **Safety of action:** Non-readonly, non-idempotent tools need caps because a bad calendar invite is worse than a wrong sentence.
- **Discovery not hardcoding:** `tools.discover(namespace="claw")` lets new registrations appear without editing the loop each time.
- **MCP interoperability:** Web search via MCP shows swappable providers behind the same registration pattern.

SECTION 6

Guardrails

- **Defense in depth:** Input topic policy, tool argument schema and counts, and output filtering address jokes that become 500-person meetings or leaked PII in tickets.
- **Input blocks save spend:** Rejected medical or legal requests never reach retrieval or generation.
- **Tool argument gate:** Constraints like future-only meetings, attendee limits, and per-session ticket counts run before Execute.
- **Blocked tools as observations:** Denied calls return tool messages so the model explains constraints instead of crashing the loop.
- **`safety_flags`:** Frontend can signal when output policy redacted or hedged the assistant text.

SECTION 7

Context assembly and the complete loop

- **Six competing sources:** System prompt, user memories, retrieved docs, tool schemas, history, and the current message share a finite token budget.
- **`assemble_context`:** Explicit reservation and trimming—often biasing documents for factual grounding while history fills slack—implements context engineering in code.
- **Complete handler:** Input guardrails, session load, assembly, agent loop with arg policy, output filter, and session append appear in one listing as Claw's harness.
- **Streaming variant:** `chat_stream` yields tokens; `filter_output_stream` buffers short spans so policy runs before users see unsafe text.
- **Terminology:** Prompt engineering writes instructions; context engineering chooses what fits; harness engineering is the loop, tools, guardrails, and integration around the model.

SECTION 8

Observability, experimentation, and deployment

- **Automatic spans:** Every `platform.*` crossing records latency, tokens, cost, and outcomes—Sarah does not hand-instrument each hop.
- **Quality scorers:** Automated retrieval-usage checks plus sampled judge scores reveal grounding skips and weak answers traces alone cannot show.
- **Prompt A/B:** Sticky user assignment compares tool-first versus behavior-first prompt structure on quality, tool accuracy, cost, and latency before promotion.
- **`@workflow` deploy contract:** Replicas, resources, timeout, stream mode, and guardrail names drive `platform deploy` through image build, registry, and gateway registration.
- **Composition example:** Async `claw-research` uses `call_parallel` across indexes and web, then synthesizes—fan-out/fan-in without bloating the chat workflow.

SECTION 9

What the platform changed versus the prototype

- **Problem-to-service table:** Memory, RAG, tools, safety, visibility, experiments, and scale each map to a dedicated service instead of ad hoc scripts.
- **~200 lines vs months:** The same capabilities that took a prototype team endless glue become composition over stable SDK clients.
- **Sam vs Sarah:** The gap is not individual skill but middleware that turns demos into operable systems with cost, safety, and measurement built in.
- **Reuse for the next app:** Customer support, code review, or summarization assistants repoint the same services and add only domain logic.
- **Claw as proof:** One user message demonstrates a unified trace, policy-enforced action, grounded text, and a deployable unit the diagram alone cannot run.